

LOOP ENGINEERING

- Do prompt ao sistema

Como construir agentes de IA que executam, verificam, corrigem e continuam trabalhando até atingir um resultado definido.



O que você vai aprender

Este guia explica o conceito de loop engineering e mostra, de forma prática, onde usar cada comando, como estruturar um Loop Charter e quais limites tornam a autonomia mais segura.

01

O prompt não acabou

Do comando isolado ao sistema

02

O que é um loop de IA

A lógica de execução, verificação e repetição

03

Onde usar cada comando

Terminal, /goal, /loop e rotinas

04

Os cinco componentes

Automação, worktrees, skills, conectores e subagentes

05

Objetivos verificáveis

Critérios claros para resultados confiáveis

06

Loop Charter na prática

Onde colar, como salvar e como reutilizar

07

Modelo completo

Um charter pronto para adaptar

08

Limites e começo seguro

Quando não usar e como ampliar a autonomia

09

Conclusão

Do prompt ao processo executável

Leitura recomendada: acompanhe os exemplos na ordem e teste primeiro em um projeto pequeno, com critérios objetivos e permissões limitadas.

Do prompt ao sistema: como funciona o Loop Engineering

O prompt não desaparece. Ele passa a fazer parte de um sistema capaz de encontrar o trabalho, executar, verificar e continuar.

Durante os últimos anos, uma das habilidades mais valorizadas em inteligência artificial foi a capacidade de escrever bons prompts. Aprendemos a fornecer contexto, explicar tarefas, definir formatos e corrigir respostas até chegar ao resultado desejado.

Essa habilidade continua sendo importante, mas uma mudança mais profunda já começou. O prompt não está desaparecendo; ele está deixando de ser uma instrução isolada e passando a fazer parte de um sistema maior.

Em vez de conversar com a inteligência artificial a cada nova etapa, você pode construir um processo no qual o agente encontra o trabalho, executa, verifica o resultado, corrige os erros, registra o progresso e continua até atingir uma condição previamente definida.

É essa mudança que vem sendo chamada de **loop engineering**: projetar processos em que a IA encontra o trabalho, executa, verifica e continua até atingir um resultado definido.

O prompt não acabou. Ele mudou de função

No uso tradicional da inteligência artificial, você normalmente entrega uma instrução e espera uma resposta. Depois, analisa o resultado, identifica um problema, envia uma correção e aguarda novamente.

Imagine que você queira revisar vinte páginas de um site. Com prompts isolados, o processo costuma funcionar assim: você pede a revisão da primeira página, confere o resultado, solicita mudanças, aprova e passa para a próxima. A IA executa uma parte do trabalho, mas você continua responsável por decidir cada passo.

Nesse modelo, você ainda precisa identificar o que falta, selecionar a próxima tarefa, lembrar o que já foi concluído, verificar a qualidade e determinar quando o projeto terminou. A inteligência artificial trabalha, mas você permanece no centro da operação, coordenando cada movimento.

O loop altera essa dinâmica. Em vez de dizer o que o agente deve fazer a cada momento, você define antecipadamente como o processo inteiro deve funcionar.

A principal diferença pode ser resumida assim:

IDEIA CENTRAL

No prompt tradicional, você controla cada tarefa. No loop, você constrói o sistema que controla as tarefas.

Isso não significa que o prompt deixou de existir. Significa que ele deixa de ser apenas uma solicitação e passa a funcionar como uma espécie de manual operacional para o agente.

A LÓGICA QUE MANTÉM O AGENTE TRABALHANDO

02

O que é um loop de IA

Um loop é um ciclo de execução que se repete até que uma condição seja atendida.

Em vez de receber uma tarefa única, responder e parar, o agente recebe um objetivo, regras de funcionamento e critérios de conclusão. Depois, ele executa sucessivas etapas até que o resultado possa ser comprovado.

Um loop confiável normalmente segue cinco movimentos.

● Encontrar o próximo trabalho

Primeiro, o agente precisa saber onde procurar o que ainda falta fazer. Essa fonte pode ser uma pasta, uma lista de tarefas, um arquivo `TODO.md`, um quadro de projetos, uma caixa de entrada ou um sistema conectado.

Por exemplo, ele pode procurar arquivos que ainda não foram revisados, testes que continuam falhando, páginas sem metadescrições ou tarefas marcadas como pendentes.

● Executar uma tarefa

Depois de encontrar o trabalho, o agente escolhe um item e o executa. Em processos confiáveis, é preferível concluir um item por vez em vez de tentar modificar tudo simultaneamente.

Isso facilita a verificação, reduz conflitos e permite identificar com clareza o que provocou um eventual erro.

● Verificar o resultado

A execução não deve ser considerada concluída apenas porque o agente produziu algo. É necessário apresentar uma evidência de que o resultado atende aos critérios definidos.

Essa verificação pode ser objetiva: os testes passaram, o link abriu, o texto respeitou o limite, todos os campos foram preenchidos ou a compilação terminou sem erros.

O funcionamento interno do Claude Code também segue uma lógica agentiva de reunir contexto, agir, verificar e repetir quando necessário.

● Corrigir o que falhou

Quando a verificação indica que o resultado ainda não está correto, o agente identifica o problema, realiza uma nova tentativa e testa novamente.

É nesse ponto que o loop substitui boa parte do trabalho manual. Você não precisa escrever repetidamente “ainda está errado” ou “tente novamente”. A própria falha na validação inicia o próximo ciclo.

● Registrar e continuar

Por fim, o agente registra o que foi concluído, o que ficou bloqueado, quais alterações foram realizadas e qual tarefa deve ser processada depois.

Se ainda houver trabalho, o ciclo começa novamente. Se todos os critérios forem atendidos, o processo para e apresenta um relatório.

A estrutura completa pode ser resumida assim:

O CICLO

Encontrar → executar → verificar → corrigir → registrar → continuar ou parar.

● A diferença entre uma tarefa e um processo

Um prompt comum descreve uma ação:

PROMPT ISOLADO

Revise esta página e melhore o texto.

Um loop descreve uma operação:

OPERAÇÃO EM LOOP

Encontre todas as páginas ainda não revisadas, trabalhe em uma por vez, preserve a proposta original, valide cada texto com o checklist definido, registre o resultado e continue até que todas estejam aprovadas ou bloqueadas.

No primeiro caso, você pede um resultado.

No segundo, você define uma forma de trabalhar.

Essa é a mudança de mentalidade mais importante. **Prompt engineering** continua sendo útil para criar boas instruções. **Loop engineering** usa essas instruções para construir processos que podem continuar funcionando sem depender de uma nova intervenção a cada etapa.

Antes de começar: onde esses comandos são utilizados

Claude Code pode funcionar no Terminal, em ambientes de desenvolvimento, no aplicativo para computador e no navegador. Neste guia, usaremos o Terminal, porque é a forma mais clara de diferenciar os comandos do computador das instruções enviadas ao agente.

Existem dois ambientes diferentes:

- O Terminal normal do seu computador.
- A interface do Claude Code aberta dentro desse Terminal.

Essa distinção é fundamental.

No Terminal normal, você usa comandos como:

TERMINAL

```
cd /caminho/do/projeto
claude
```

O primeiro comando entra na pasta do projeto. O segundo inicia o Claude Code.

Depois que o Claude Code estiver aberto, você não está mais digitando comandos diretamente para o sistema operacional. Está conversando com o agente dentro do contexto daquele projeto.

É nessa interface que você utiliza instruções como:

DENTRO DO CLAUDE CODE

```
/goal ...
/loop ...
/revisar-roteiros
```

Portanto, `/goal` e `/loop` não devem ser digitados no Terminal antes de abrir o Claude Code. Eles são usados dentro de uma sessão ativa do Claude Code.

• `/goal`: trabalhar até alcançar uma condição

O comando `/goal` é utilizado quando existe uma linha de chegada verificável.

Você informa uma condição final, e Claude continua trabalhando ao longo de vários turnos sem exigir que você escreva "continue" depois de cada etapa. Ao final de cada turno, um modelo avaliador separado verifica se a condição definida já foi atendida. Caso ainda não tenha sido, Claude começa outro turno.

A documentação atual informa que `/goal` exige Claude Code 2.1.139 ou posterior. Para verificar a versão instalada, use o Terminal normal:

TERMINAL

```
claude --version
```

Depois, entre na pasta desejada e abra o Claude Code:

TERMINAL

```
cd /caminho/do/projeto  
claude
```

Já dentro do Claude Code, você pode escrever:

CLAUDE CODE

```
/goal todos os arquivos da pasta /reels possuem um gancho claro, uma explicação  
compreensível, um exemplo concreto e um CTA final. Valide cada arquivo antes de considerá-lo  
concluído e pare após 30 arquivos ou 20 turnos.
```

O objetivo começa imediatamente. Não é necessário enviar outro prompt depois dele.

Para consultar o andamento, digite dentro do Claude Code:

CLAUDE CODE

```
/goal
```

Para interromper antes que a condição seja alcançada:

CLAUDE CODE

```
/goal clear
```


A condição deve ser algo que possa ser demonstrado na própria execução. O avaliador não abre arquivos nem executa comandos sozinho; ele analisa as evidências que Claude apresentou na conversa. Por isso, “deixe o projeto perfeito” é uma condição ruim, enquanto “todos os testes passam e `git status` está limpo” é verificável.

● **`/loop`: repetir uma tarefa em determinado intervalo**

O comando `/loop` é usado quando o processo não precisa avançar continuamente até uma conclusão, mas deve ser repetido em determinados intervalos.

Ele é adequado para acompanhar uma implantação, verificar se um site voltou ao ar, consultar periodicamente uma fila ou lembrar Claude de executar uma ação mais tarde.

Primeiro, abra Claude Code na pasta adequada:

TERMINAL

```
cd /caminho/do/projeto
claude
```

Depois, já dentro do Claude Code, escreva:

CLAUDE CODE

```
/loop 30m verifique se o site voltou a funcionar. Quando a página inicial carregar normalmente, me avise e interrompa esta verificação.
```

Nesse caso, Claude repetirá a tarefa a cada trinta minutos dentro da sessão.

A diferença prática é simples:

`/goal` inicia um novo turno assim que o anterior termina e continua até uma condição ser atendida.

`/loop` espera o intervalo definido antes de executar novamente.

As tarefas criadas com `/loop` pertencem à sessão atual. Elas exigem que a máquina e a sessão estejam disponíveis, embora possam ser restauradas por um período quando a conversa é retomada. Para processos que precisam funcionar independentemente do computador, existem alternativas como rotinas em nuvem, tarefas agendadas pelo aplicativo para computador e GitHub Actions.

● **Rotinas: quando o processo precisa funcionar com o computador desligado**

Um `/loop` executado no Terminal não é a mesma coisa que uma automação permanente.

Quando o processo precisa continuar mesmo com o notebook fechado, uma opção é criar uma rotina. Ela combina uma instrução, um ou mais repositórios e os conectores necessários. Pode ser acionada por horário, chamada de API ou evento do GitHub e executada em infraestrutura gerenciada. Esse tipo de recurso pode evoluir ao longo do tempo.

Um exemplo seria uma rotina que, todas as manhãs, analisa novas tarefas de um projeto, classifica cada uma, identifica quais exigem atenção e publica um resumo em uma ferramenta conectada.

A escolha correta depende do objetivo:

- Use `/goal` quando houver um resultado final mensurável.
- Use `/loop` para verificações periódicas enquanto a sessão estiver ativa.
- Use uma rotina quando o processo precisar ser recorrente e independente do computador.

Os cinco componentes que tornam um loop mais forte

Os comandos são apenas o mecanismo que mantém a execução acontecendo. Para criar um sistema realmente útil, o agente também precisa de contexto, isolamento, ferramentas e revisão.

● 1. Automação

A automação define quando o processo começa ou recomeça.

Ela pode ser simples, como um `/loop` executado durante uma sessão, ou mais permanente, como uma rotina na nuvem. Também pode ser acionada por eventos, scripts ou ações externas.

A automação não melhora a qualidade por si só. Ela apenas reduz a necessidade de iniciar manualmente cada execução. Um processo ruim executado automaticamente continua sendo um processo ruim, apenas mais rápido.

● 2. Worktrees

Quando dois agentes trabalham ao mesmo tempo na mesma pasta, ambos podem editar os mesmos arquivos e provocar conflitos.

Git worktrees permitem que cada sessão trabalhe em uma cópia isolada do projeto, associada a uma branch separada. Assim, um agente pode desenvolver uma funcionalidade enquanto outro corrige um problema diferente sem que as alterações se misturem.

No Terminal normal, você pode iniciar uma sessão isolada com:

```
TERMINAL - SESSÃO 1
```

```
claude --worktree feature-auth
```

Em outro Terminal, pode iniciar outra:

```
TERMINAL - SESSÃO 2
```

```
claude --worktree corrigir-login
```

Cada sessão trabalhará em seu próprio checkout. Esse recurso é útil para operações paralelas, mas não é obrigatório quando apenas um agente está trabalhando.

● 3. Skills

Uma skill é um arquivo que contém um procedimento reutilizável.

Ela serve para instruções que você costuma repetir: como revisar um conteúdo, como testar uma aplicação, quais critérios utilizar ou qual formato seguir.

Em vez de colar o mesmo manual em todas as conversas, você salva o procedimento uma vez e chama a skill quando necessário.

Para criar uma skill específica de um projeto, a estrutura é:

ESTRUTURA DO PROJETO

```
meu-projeto/  
├── .claude/  
├── skills/  
├── revisar-roteiros/  
└── SKILL.md
```

O nome da pasta se transforma no comando. Portanto:

CAMINHO DA SKILL

```
.claude/skills/revisar-roteiros/SKILL.md
```

cria o comando:

CLAUDE CODE

```
/revisar-roteiros
```

As skills podem ser chamadas diretamente ou carregadas automaticamente quando a descrição delas corresponde à tarefa. Elas são apropriadas quando você continua colando o mesmo checklist ou procedimento nas conversas.

● 4. Conectores

Sem conectores, o agente trabalha principalmente com os arquivos e ferramentas disponíveis no ambiente local.

Com MCP, Claude Code pode se conectar a serviços externos, bancos de dados, sistemas de monitoramento, gestores de tarefas e outras aplicações. Isso permite que “encontrar o trabalho” signifique consultar uma ferramenta real, e não apenas abrir um arquivo local.

Por exemplo, um loop conectado pode:

- consultar tarefas abertas em um gerenciador de projetos
- ler alertas de um sistema de monitoramento
- buscar informações em uma base de dados
- criar rascunhos em uma ferramenta autorizada
- atualizar o status de uma tarefa após a validação.

Os conectores devem ser configurados com cuidado. O agente só deve receber as permissões necessárias, principalmente quando existe a possibilidade de enviar mensagens, modificar dados ou executar ações irreversíveis.

● 5. Subagentes

O agente que executou uma tarefa pode não ser a melhor pessoa para avaliar o próprio trabalho. Ele conhece as decisões tomadas e pode repetir os mesmos erros durante a revisão.

Subagentes permitem separar funções. Um agente pode implementar, enquanto outro atua como revisor independente. Cada subagente pode possuir seu próprio contexto, modelo, conjunto de ferramentas e permissões.

Para criar um subagente específico do projeto, você pode pedir dentro do Claude Code:

PROMPT PARA CRIAR O SUBAGENTE

Crie um subagente chamado revisor em `.claude/agents/`. Ele deve trabalhar somente em modo de leitura, analisar as alterações realizadas pelo agente principal e verificar qualidade, segurança, testes e respeito às regras do projeto. Ele não pode modificar arquivos.

Claude criará um arquivo semelhante a:

ARQUIVO CRIADO

```
.claude/agents/revisor.md
```

Depois, você pode solicitar:

CLAUDE CODE

Use o subagente revisor para analisar as alterações antes de considerar esta tarefa concluída.

Os subagentes personalizados são armazenados como arquivos Markdown em `.claude/agents/` para um projeto ou em `~/ .claude/agents/` para uso em todos os projetos.

SEM CRITÉRIO DE QUALIDADE NÃO EXISTE LOOP CONFIÁVEL

05

O elemento que conecta tudo: um objetivo verificável

Nenhum desses recursos resolve o principal problema de um loop mal construído: a ausência de uma definição clara de qualidade.

Um agente só consegue continuar trabalhando com segurança quando existe uma forma objetiva de verificar o resultado.

Compare estas duas instruções:

OBJETIVO VAGO

Melhore todos os artigos.

OBJETIVO VERIFICÁVEL

Revise todos os artigos da pasta /blog. Cada título deve ter no máximo 60 caracteres, cada descrição deve ter no máximo 160, nenhum link pode retornar erro e o corpo do texto não pode ser alterado. Registre as contagens e o resultado da verificação de cada arquivo.

A primeira instrução depende de uma opinião subjetiva.

A segunda define critérios que podem ser medidos.

Antes de construir qualquer loop, faça a seguinte pergunta:

A PERGUNTA QUE DEFINE O LOOP

Qual evidência demonstra que este trabalho foi concluído corretamente?

Se não houver uma resposta clara, a tarefa ainda não está pronta para ser executada de forma autônoma.

● Exemplo prático: revisar uma pasta de roteiros

Imagine uma pasta com cinquenta roteiros de Reels.

O objetivo é garantir que todos tenham um gancho forte, explicação compreensível, exemplo concreto, conclusão e CTA.

Um prompt isolado poderia revisar um roteiro. Um loop pode executar o processo completo.

Primeiro, o agente procura quais arquivos ainda não foram aprovados. Depois, abre o primeiro roteiro, compara o texto com os critérios, corrige somente o necessário e estima o tempo de fala.

Em seguida, verifica novamente. Se algum critério continuar ausente, faz uma nova tentativa. Depois da aprovação, registra o resultado e passa para o próximo arquivo.

O sistema pode continuar até que todos os roteiros estejam aprovados, até que determinada quantidade seja processada ou até que apareça uma situação que exija uma decisão humana.

A vantagem não está apenas na velocidade. Todos os itens passam pela mesma metodologia, o que aumenta a consistência do resultado.

O que é um Loop Charter

Loop Charter não é um comando oficial do Claude Code.

É o nome dado, neste guia, a um documento que descreve como o agente deve conduzir determinado processo. Em português, ele poderia ser chamado de carta operacional do loop.

O charter explica:

- qual é o objetivo
- onde encontrar o trabalho
- como executar
- como verificar
- o que registrar
- quais ações são proibidas
- quando continuar
- quando parar.

Ele pode ser utilizado de três formas: colado diretamente dentro do Claude Code, salvo como um arquivo do projeto ou transformado em uma skill.

• Como usar o Loop Charter diretamente no Claude Code

Essa é a forma mais simples para testar.

No Terminal normal, entre na pasta e abra o Claude Code:

TERMINAL

```
cd /caminho/do/projeto
claude
```

Depois que a interface estiver aberta, cole o charter como uma mensagem:

LOOP CHARTER - EXEMPLO COMPLETO

Você está executando um processo contínuo, e não respondendo a uma tarefa isolada.

OBJETIVO

Revisar todos os arquivos da pasta /reels até que cada roteiro possua um gancho claro, uma explicação compreensível, um exemplo concreto e um CTA coerente.

ONDE ENCONTRAR O TRABALHO

Leia todos os arquivos .md existentes na pasta /reels. Considere pendente qualquer arquivo que ainda não esteja registrado como aprovado em LOOP-STATE.md.

COMO EXECUTAR

Trabalhe em um arquivo por vez. Preserve a ideia central e altere somente o necessário. Não invente fatos, estudos ou dados. Não modifique arquivos fora da pasta /reels.

COMO VERIFICAR

Um roteiro só pode ser aprovado quando:

- o gancho estiver presente no início;
- a explicação puder ser compreendida por alguém que não conhece o tema;
- existir pelo menos um exemplo concreto;
- houver uma conclusão;
- o CTA estiver relacionado ao conteúdo;
- o tempo estimado de fala não ultrapassar 60 segundos.

Quando algum critério falhar, corrija e verifique novamente. Faça no máximo três tentativas por arquivo. Depois disso, registre o item como bloqueado.

COMO REGISTRAR

Use LOOP-STATE.md. Após cada arquivo, registre o nome, o status, as alterações realizadas, o resultado da validação e qualquer decisão humana necessária.

QUANDO PARAR

Pare quando todos os arquivos estiverem aprovados ou bloqueados, ou após processar 20 arquivos nesta execução. Ao final, apresente um relatório curto.

Ao enviar essa mensagem, Claude pode começar a executar o processo.

Essa abordagem é adequada para um teste ou uma operação que provavelmente não será repetida muitas vezes.

● Como salvar o Loop Charter em um arquivo

Quando o processo será usado novamente, é melhor salvar o charter em um arquivo.

A estrutura pode ser:

ESTRUTURA DO PROJETO

```
meu-projeto/  
├─ CLAUDE.md  
├─ LOOP-CHARTER.md  
├─ LOOP-STATE.md  
└─ reels/
```

O papel de cada arquivo é diferente.

`CLAUDE.md` contém regras permanentes do projeto, como padrões técnicos, proibições e convenções.

`LOOP-CHARTER.md` descreve o processo específico que será executado.

`LOOP-STATE.md` armazena o progresso, os resultados e os bloqueios.

Depois de criar os arquivos, abra o Claude Code no Terminal:

TERMINAL

```
cd /caminho/do/projeto  
claude
```

Já dentro do Claude Code, escreva:

CLAUDE CODE

```
Leia LOOP-CHARTER.md, confirme que compreendeu as regras e execute o processo. Atualize LOOP-STATE.md depois de cada item.
```

Nesse formato, o agente pode reutilizar as mesmas instruções sem que você precise colar o texto novamente.

Claude Code também possui memória automática para registrar conhecimento útil sobre o projeto entre sessões, mas um arquivo de estado explícito continua sendo útil quando você precisa acompanhar com precisão quais itens foram processados.

● Como combinar o charter com `/goal`

O charter descreve como trabalhar.

O `/goal` define até quando continuar.

Primeiro, abra o Claude Code:

TERMINAL

```
cd /caminho/do/projeto
claude
```

Depois, dentro da sessão, digite:

CLAUDE CODE

```
/goal execute o processo definido em LOOP-CHARTER.md até que todos os arquivos da pasta /
reels estejam aprovados ou registrados como bloqueados em LOOP-STATE.md. Demonstre no
relatório final que cada critério foi verificado. Não modifique arquivos fora de /reels e
pare após 20 turnos.
```

Claude começa imediatamente e continua até que o avaliador considere a condição atingida ou até que o limite definido seja alcançado.

Essa combinação é especialmente útil porque separa três responsabilidades:

- O charter contém o método.
- O arquivo de estado contém o progresso.
- O `/goal` mantém a execução ativa até a linha de chegada.

• Como transformar o charter em uma skill

Quando você pretende repetir o mesmo processo com frequência, a melhor solução é transformá-lo em uma skill.

Dentro do projeto, crie:

CAMINHO DA SKILL

```
.claude/skills/revisar-roteiros/SKILL.md
```

O conteúdo pode ser:

SKILL.MD

```
---
description: Revisa os roteiros da pasta /reels usando o padrão editorial do projeto. Use
quando for necessário analisar, corrigir ou validar roteiros.
---
```

```
# Processo de revisão
```

Leia as regras do projeto em CLAUDE.md e o progresso em LOOP-STATE.md.

Trabalhe em um roteiro por vez. Cada roteiro deve conter:

- gancho claro;
- explicação compreensível;
- exemplo concreto;
- conclusão;
- CTA coerente;
- tempo estimado de até 60 segundos.

Não invente informações. Não altere arquivos fora de /reels.

Após cada revisão, atualize LOOP-STATE.md com o nome do arquivo, as alterações, o resultado e os bloqueios.

Faça no máximo três tentativas por arquivo.

Depois, abra Claude Code e execute:

```
CLAUDE CODE
```

```
/revisar-roteiros
```

Também é possível combinar a skill com `/goal`:

```
CLAUDE CODE
```

```
/goal use a skill /revisar-roteiros até que todos os arquivos da pasta /reels estejam aprovados ou bloqueados e o resultado esteja documentado em LOOP-STATE.md. Pare após 20 turnos.
```

A skill contém o procedimento reutilizável. O `/goal` mantém a execução acontecendo até que a condição final seja alcançada.

Um modelo completo para adaptar

O modelo abaixo deve ser colado dentro do Claude Code, salvo como `LOOP-CHARTER.md` ou transformado em uma skill. Não deve ser executado diretamente como um comando do sistema operacional.

MODELO DE LOOP CHARTER

Você está executando um processo contínuo, e não respondendo a uma tarefa isolada.

OBJETIVO FINAL

[Descreva o estado final que precisa ser alcançado. Use algo verificável.]

ONDE ENCONTRAR O TRABALHO

[Informe onde estão as tarefas: pasta, arquivo, lista, banco, quadro ou ferramenta conectada.]

MODO DE EXECUÇÃO

Trabalhe em um item por vez. Conclua e verifique o item atual antes de começar o próximo.

Respeite os padrões existentes no projeto. Não invente estruturas novas sem necessidade.

Não realize ações irreversíveis, como excluir, publicar, enviar, comprar, implantar ou modificar dados de produção, sem autorização explícita.

CRITÉRIOS DE VALIDAÇÃO

Um item só pode ser considerado concluído quando:

- [critério verificável 1]
- [critério verificável 2]
- [critério verificável 3]

Apresente evidências da validação. Confiança não é evidência.

CORREÇÃO

Quando um critério falhar, identifique a causa, corrija e valide novamente.

Faça no máximo [N] tentativas por item. Depois desse limite, registre o item como bloqueado e continue somente quando isso for seguro.

MEMÓRIA DO PROCESSO

Use `LOOP-STATE.md`.

Depois de cada item, registre:

```
- nome;  
- status;  
- alterações;  
- resultado da verificação;  
- bloqueios;  
- decisões humanas necessárias.
```

Leia esse arquivo no início de cada execução para não repetir itens já concluídos.

LIMITES

Proibido alterar:

[Liste arquivos, sistemas e áreas protegidas.]

Exigir autorização antes de:

[Liste ações sensíveis.]

Máximo de [N] itens ou [N] turnos por execução.

CONDIÇÃO DE PARADA

Pare quando todos os itens estiverem concluídos ou bloqueados, quando o limite da execução for atingido ou quando surgir uma decisão que não possa ser tomada com segurança.

Ao final, entregue um relatório com o que foi concluído, o que falhou, o que ficou bloqueado e quais decisões precisam de intervenção humana.

● Por que o arquivo de estado é tão importante

Sem um registro de progresso, cada nova execução corre o risco de começar do zero.

O agente pode revisar novamente itens já concluídos, esquecer por que uma tarefa ficou bloqueada ou repetir uma tentativa que já falhou.

Um arquivo como `LOOP-STATE.md` cria continuidade.

Ele pode ter esta estrutura:

```
# Estado do loop

## roteiro-01.md

Status: concluído
Alterações: gancho reduzido e CTA substituído
Validação: 6 de 6 critérios atendidos
Tentativas: 2

## roteiro-02.md

Status: bloqueado
Motivo: afirmação médica sem fonte disponível
Necessita decisão humana: sim
```

O estado não precisa ser sofisticado. Ele precisa ser claro, atualizado e lido no início de cada execução.

LIMITES, CUSTOS E SEGURANÇA

08

Quando não utilizar loops

Nem toda tarefa precisa de um sistema autônomo.

Escrever um e-mail, resumir um documento curto ou responder uma pergunta isolada geralmente é mais rápido com um prompt simples.

Loops fazem mais sentido quando o trabalho é repetitivo, contém vários itens, segue regras estáveis e possui critérios verificáveis.

Também existe um custo. Um loop que executa, verifica, corrige e tenta novamente consome mais turnos e mais tokens do que uma resposta única. Por isso, é prudente começar com poucos itens, acompanhar a primeira execução e definir limites.

Outra limitação é a segurança. Um loop com permissão para excluir arquivos, publicar conteúdo, enviar mensagens ou modificar produção pode ampliar rapidamente um erro. As primeiras versões devem funcionar com o menor conjunto possível de permissões.

A regra central é esta:

REGRA CENTRAL

Um loop sem verificação não automatiza qualidade. Ele apenas automatiza erros.

● Como começar sem complicar

Escolha um processo pequeno e incômodo que você já executa manualmente. Não comece com toda a empresa, todo o projeto ou acesso irrestrito às ferramentas.

Pode ser uma pasta com cinco arquivos, uma pequena lista de tarefas ou um conjunto de links para verificar.

Defina o resultado final, estabeleça três ou quatro critérios objetivos, limite a execução e acompanhe o primeiro teste.

Somente depois que o comportamento estiver previsível, transforme o charter em skill, adicione um subagente revisor, conecte ferramentas ou coloque o processo em uma rotina.

A sequência mais segura é:

SEQUÊNCIA MAIS SEGURA

observar manualmente → testar com poucos itens → verificar → corrigir o processo → ampliar a autonomia.

O PROMPT CONTINUA EXISTINDO, MAS DEIXA DE COMANDAR CADA ETAPA

09

Conclusão

O avanço mais importante na utilização de agentes de IA não está em abandonar os prompts.

Está em parar de depender de um novo prompt a cada etapa.

Quando objetivo, contexto, ferramentas, verificação, memória e limites são organizados dentro de um mesmo processo, a inteligência artificial deixa de ser apenas uma interface que responde perguntas e começa a funcionar como parte de uma operação.

Prompt engineering ensina a dar boas instruções.

Loop engineering ensina a construir um sistema que sabe quando executar, como verificar, quando corrigir e em qual momento parar.

DO PROMPT AO SISTEMA

Loop engineering

O futuro não será definido apenas por quem conversa melhor com a inteligência artificial, mas por quem consegue transformar conhecimento, regras e critérios em processos executáveis.

IA não é apenas skill. É processo, estrutura e critério.

@bender.ia

